Wyszukiwanie sekwencyjne

Stosowane gdy zbiór w którym należy realizować wyszukiwanie elementu nie jest uporządkowany.

```
bool search(double value, double data[], int size) {
  for (int i = 0; i < size; ++i) {
    if (data[i] == value) {
       cout << ”Wartość znaleziona na pozycji ”<< i << endl;
       return true;
    }
  }
  return false;
}
```

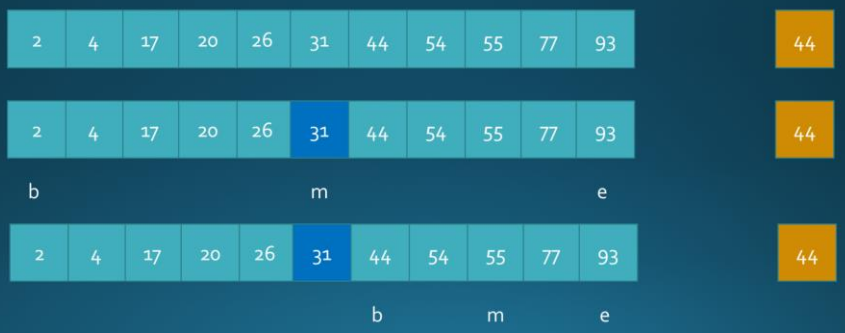
Wyszukiwanie sekwencyjne

Złożoność obliczeniowa: O(n)

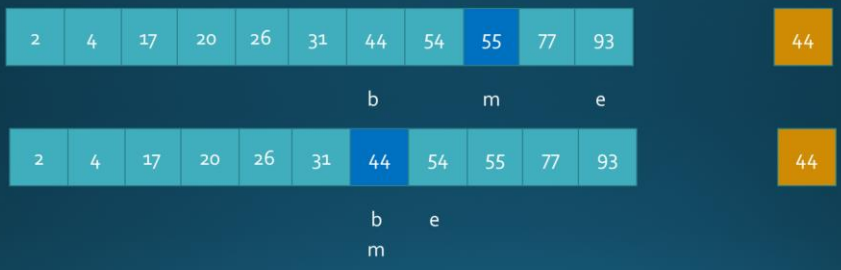
Średnia liczba kroków po której zostanie znaleziona wartość: N/2

Wyszukiwanie binarne

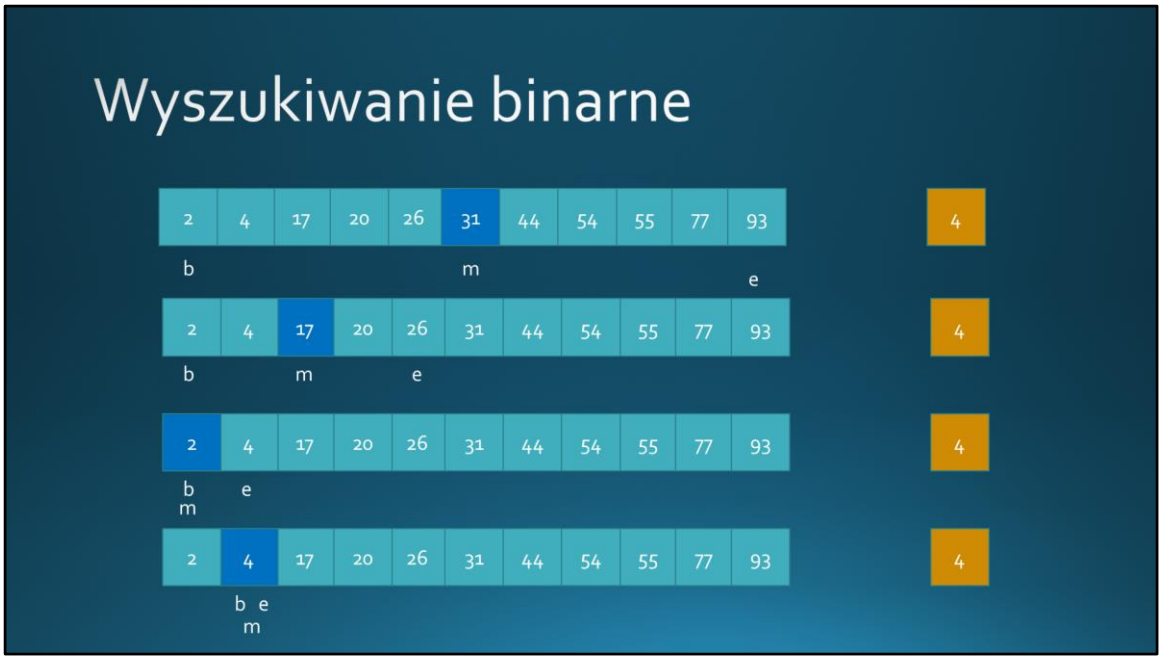
Stosowane gdy zbiór jest uporządkowany rosnąco bądź malejąco.



Wyszukiwanie binarne



Wyszukiwanie binarne



Wyszukiwanie binarne

```
bool search(double value, double data[], int size) {
  int begin = 0;
  int end = size - 1;
  while(begin <= end) {
    int middle = (begin + end) / 2;
    if (data[middle] == value) {
      cout << "Wartość znaleziona na pozycji " << middle << endl;
      return true;
    }
    if (data[middle] > value)
       end = middle - 1;
    else
       begin = middle + 1;
  }
  return false;
}
```

Wyszukiwanie binarne

Liczba przejrzanych elementów nigdy nie przekracza liczby

$$\lfloor \log_2 N \rfloor + 1$$

Złożoność obliczeniowa: $O(\log n)$

Tablice asocjacyjne…

… inaczej kodowanie mieszające, hashing

Tablica asocjacyjna to tablica do której danych sięga się za pomocą specjalnego indeksu zwanego **kluczem**.

Do wyszukiwania danej wymagana jest **funkcja mieszająca** której argumentem jest klucz a która zwraca **kod mieszania**.

Klucze mogą być dowolnego typu.

Kody mieszania są zawsze liczbami całkowitymi.

Tablice asocjacyjne

Niech, dla uproszczenia daną do wstawienia do tablicy asocjacyjnej będzie sam klucz k.

```
int hash_table[97];

int index = h(k); // h –jest funkcją mieszającą
hash_table[indeks] = k;
```

Miejsce wstawienia danej zależy pośrednio od samej danej.

Tablice asocjacyjne

Miejsce wstawienia danej zależy pośrednio od samej danej.



Wyszukanie miejsca do wstawienia lub wyszukanie danej w tablicy jest więc natychmiastowe.

Czy liczba kluczy jest mniejsza od liczby komórek w tablicy?



Nie, zbiór kluczy jest znacznie większy.

Tablice asocjacyjne

Zbiór kluczy jest znacznie większy od zbioru indeksów tablicy.



Mogą się zdarzać, i zdarzają się kolizje.

Dwa lub więcej kluczy może być odwzorowanych przez funkcję mieszającą w ten sam indeks.

Dobra funkcja mieszająca minimalizuje prawdopodobieństwo kolizji, ale mechanizm ich rozwiązywania jest i tak konieczny.

Dobór funkcji mieszającej

Funkcja modularna:

- $h(k) = k \bmod M$,
- M jest liczbą pierwszą.

Funkcja z mnożeniem:

- $h(k) = \lfloor M(kA - \lfloor kA \rfloor) \rfloor$,
- $0 < A < 1$, zwykle $A \approx 0{,}618$.
- przykład: M=2000, k=6321

$$h(k) = \lfloor 2000(6321 * 0{,}618 - \lfloor 6321 * 0{,}618 \rfloor \rfloor$$
$$h(k) = \lfloor 2000(3906{,}378 - \lfloor 3906{,}378 \rfloor \rfloor = \lfloor 2000 * 0{,}378 \rfloor = 756$$

Dobór funkcji mieszającej

$h(k) = k \bmod M,$

	97	100
16273	74	73
223	29	23
17	17	17
2132	95	32
73223	85	23
1117	50	17
115	18	15

Dobór wartości M ma kluczowe znaczenie.

Dobór funkcji mieszającej

$h(k) = k \bmod M,$

Wartość M nie powinna być potęgą 10 bo wartością h(k) stają się najmłodsze cyfry liczby k.

Wartość M nie powinna być potęgą 2 bo wartością h(k) stają się najmniej zanczące bity liczby k.

M powinno być liczbą pierwszą, najlepiej niezbyt bliską potędze 2.

Wyznaczanie klucza

Do tablicy asocjacyjnej chcemy wstawiać słowa. Jak na podstawie słowa wyznaczyć klucz k?

k = liczba liter w słowie

kot – k = 3
maj – k = 3
tom – k = 3

Słowa o tej samej liczbie liter będą miały ten sam klucz → kolizja.

Wyznaczanie klucza

Do tablicy asocjacyjnej chcemy wstawiać słowa. Jak na podstawie słowa wyznaczyć klucz k?

k = wartość całkowita wyznaczona na podstawie kodów ascii

kot – k = 107 (k) + 111 (o) + 116 (t) = 334
maj – k = 109 (m) + 97 (a) + 106 (j) = 312
tom – k = 116 (t) + 111 (o) + 109 (m) = 336
tok – k = 116 (t) + 111 (o) + 107 (k) = 334

Słowa składających się z tych samych liter będą miały ten sam klucz → kolizja.

Wyznaczanie klucza

Do tablicy asocjacyjnej chcemy wstawiać słowa. Jak na podstawie słowa wyznaczyć klucz k?

k = wartość całkowita wyznaczona na podstawie kodów ascii v2

kot – $k = 107 * 128^2$ (k) + $111 * 128^1$ (o) + $116 * 128^0$ (t) = 1767412
maj – $k = 109 * 128^2$ (m) + $97 * 128^1$ (a) + $106 * 128^0$ (j) = 1798378
tom – $k = 116 * 128^2$ (t) + $111 * 128^1$ (o) + $109 * 128^0$ (m) = 1914861
tok – $k = 116 * 128^2$ (t) + $111 * 128^1$ (o) + $107 * 128^0$ (k) = 1914859

Metoda odpowiednia dla krótkich słów.

Kodowanie mieszające długich kluczy

Długi klucz – duża wartość lub długi ciąg znaków.

Należy wykorzystać schemat Hornera

$$(ax + b) \bmod M = \Big(\big((ax) \bmod M\big) + b\Big) \bmod M$$

Schemat Hornera – schemat obliczania wartości wielomianu z minimalną liczbą mnożeń

Kodowanie mieszające długich kluczy

```
char vector[100];
int Modulo = PRIME;
int h = 0;
int base = 127;

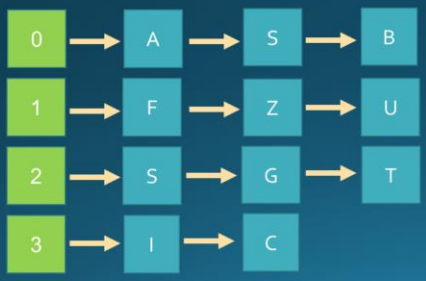
for (int i = 0; vector[i]; ++i) {
        h = (base * h + vector[i]) % Modulo;
}
```

Unikanie kolizji

- Łańcuchowe tablice asocjacyjne,
- Sondowanie liniowe,
- Podwójne mieszanie.

Łańcuchowe tablice asocjacyjne

A	S	F	G	I	S	Z	T	B	C	U
0	2	1	2	3	0	1	2	0	3	1



Łańcuchowe tablice asocjacyjne

Jakie jest prawdopodobieństwo kolizji pk?

pk = 1 – pbk (prawdopodobieństwo braku kolizji)

pbk_1 dla pierwszego klucza: 1
pbk_2 dla drugiego klucza: (N – 1) / N
pbk_3 dla drugiego klucza: (N – 2) / N
…

Łańcuchowe tablice asocjacyjne

Jakie jest prawdopodobieństwo kolizji pk?

pk = 1 – pbk (prawdopodobieństwo braku kolizji)

$$pbk = 1 * \frac{N-1}{N} * \frac{N-2}{N} * \frac{N-3}{N} * \cdots * \frac{N-n+2}{N}$$

Kolizja jest nieunikniona niezależnie od rozmiaru tablicy N

Sondowanie liniowe

Wszystkie dane zapisywane są w tej samej tablicy – otwarte adresowanie.

W przypadku kolizji obliczaj adres wtórny h_i aż do znalezienia wolnego miejsca lub znalezienia klucza.

$$h_0 = k \bmod M$$

$$h_i = (h_0 + i) \bmod M, \qquad i = 1, \dots, M-1$$

Sondowanie liniowe

Przykład:

Do tablicy wstawiane będą koleje litery: A, S, F, G, I, P, Z, T



A: h(k) = 65 mod 11 = 10



Sondowanie liniowe

Do tablicy wstawiane będą koleje litery: A, S, F, G, I, P, Z, T

S: h(k) = 83 mod 11 = 6



F: h(k) = 70 mod 11 = 4



G: h(k) = 71 mod 11 = 5



Sondowanie liniowe

Do tablicy wstawiane będą koleje litery: A, S, F, G, I, P, Z, T

I: h(k) = 73 mod 11 = 7



P: h(k) = 80 mod 11 = 3



Z: h(k) = 90 mod 11 = 2



Sondowanie liniowe

Do tablicy wstawiane będą koleje litery: A, S, F, G, I, P, Z, T

T: $h_0(k) = 84 \bmod 11 = 7$ – kolizja

$h_1(k) = (h_0(k) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$



Sondowanie liniowe

- prosty algorytm,
- brak wymagań na M,
- pierwotne zbijanie wartości – długie ciągi zajętych pozycji w tablicy.

Podwójne mieszanie

Metoda wykorzystuje dwie funkcje mieszające.

Pierwsza próba:

$$h_0 = k \bmod N$$

Jeśli występuje kolizja wyznacza się nowy klucz:

$$h_i = \big(h_0 + iH(k)\big) \bmod N, \qquad i = 1, \dots, N-1$$

gdzie

$$H(k) = (k \bmod M) + 1, \qquad M < N$$

Podwójne mieszanie

Dobrze dobrane wartości M i N są bliźniaczymi liczbami pierwszymi (różnią się między sobą o 2):

179 181
521 523
1031 1033
1667 1669
2549 2551
…

Usuwanie z tablicy asocjacyjnej

Usuwania elementu z tablicy asocjacyjnej nie można zakończyć na usunięciu znalezionego elementu z powodu kolizji podczas wstawiania.

Rozwiązania:

1. Wprowadzenie pola DELETED i FREE,
2. Ponowne mieszanie kolejnych wartości.